AUTHORIZATION			
	NAME AND TITLE	**SIGNATURE**	**DATE**
Issued by:	Natan Blanks, SVP Research & Development		
Reviewed and Approved by:	Yael Goldbrener, QA Manager		

REVISION HISTORY				
REV. #	**CO #**	**DESCRIPTION**	**PREPARED BY**	**DATE**
02.03	NA	QA-SOP-0002: Moved some detail from the Quality Manual to this SOP and from this SOP to the new work instructions relating to it.	Alexey Sorokin, Scrum Master	January 24th, 2011
03.00	010	QA-SOP-0002 – Design and Development was changed to SW-SOP-4.2.3. In addition, procedure has been adapted and revised towards the transition of QMS from Canada to Israel.	Mike Zeevi, Senior Consultant, Michael Lutsky, QA/ RA Manager	April 3rd, 2013
03.01	057	STP preparation requirements re-defined	Meny Lev-Sagie, VP, Product Development	February 24th, 2016
04.00	077	Updated to comply with IO 13485:2016; reviewed and updated the Standards list; changed the software safety classification according to IEC 62304:2015 (ed. 1.1)	Sharon Renick, QA Manager	September 13th, 2018
05.00	097	Revisions to better fit 62034 and AAMI TIR45 on Agile development; clarifications of relationship between SPMP and ECO; identification of stories related to risks; separation of FDS into separate requirement and design documents; multiple other small changes and clarifications.	Natan Blanks, SVP R&D	August 5th, 2019
05.01	99	Addition of requirements for clinical validation	Yael Goldbrener, QA Manager	August 22nd, 2019

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

DISTRIBUTION LIST		
COPY #	**NAME**	**TITLE/ DEPARTMENT**
Original	Master File	Quality Assurance
1		
2		
3		

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

Table of contents

Page

1. PURPOSE 5

2. SCOPE 5

3. DEFINITIONS AND ABBREVIATIONS 5

4. APPLICABLE DOCUMENTS 6

5. SOFTWARE DEVELOPMENT LIFE CYCLE 7

5.1. SOFTWARE DEVELOPMENT PRODUCTS 7
5.1.1. New Products 8
5.1.2. Major Modifications 8
5.1.3. New Features 8
5.1.4. Bug Fixes and Minor Changes 8
5.1.5. Utilities 8
5.2. SOFTWARE DEVELOPMENT ACTIVITIES 9
5.3. SOFTWARE DEVELOPMENT PHASES 10
5.4. SOFTWARE SUPPORT ACTIVITIES 12

6. SOFTWARE PROJECT MANAGEMENT 12

7. LEVEL OF CONCERN/SOFTWARE SAFETY CLASSIFICATION 13

8. SOFTWARE RISK MANAGEMENT 14

9. SOFTWARE REQUIREMENTS 14

9.1. SOFTWARE REQUIREMENTS SPECIFICATION 15
9.2. SOFTWARE REQUIREMENTS REVIEW 15
9.3. VERSION REQUIREMENTS SPECIFICATION 15

10. SOFTWARE DESIGN 16

10.1. HIGH LEVEL DESIGN DOCUMENT 16
10.2. INTERFACE DESIGN DOCUMENT 17
10.3. SOFTWARE DESIGN REVIEW 17
10.4. FEATURE DESIGN SPECIFICATION 17
10.5. FEATURE DESIGN REVIEW 18

11. SOFTWARE/FEATURE IMPLEMENTATION 18

11.1. DETAILED DESIGN 18
11.2. WALKTHROUGH 19
11.3. CODING 19
11.4. CODE REVIEW 19
11.5. UNIT TESTING 20
11.6. CODE CONTROL 20

12. SYSTEM TESTING 20

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

12.1. INTEGRATION TESTS 20
12.2. VERIFICATION TESTS 21
12.3. SOFTWARE VALIDATION TESTS 21
12.3.1. Software Test Plan 21
12.3.2. Software Test Description 22
12.3.3. Test Readiness Review 22
12.3.4. Validation Test Runs 22
12.3.5. Clinical Validation 23
12.3.6. Software Test Report 23

13. OTS TESTING 23

13.1. OTS SOURCE CODE COMPONENT 23
13.2. OTS SOFTWARE COMPONENT 24
13.3. OTS SOFTWARE PACKAGE 24
13.4. OTS SOFTWARE FOR USE IN THE OFFICE 24

14. SOFTWARE CONFIGURATION MANAGEMENT 25

14.1. CONFIGURATION IDENTIFICATION 25
14.2. CONFIGURATION CONTROL 25
14.3. CONFIGURATION MANAGEMENT LIBRARIES 26
14.4. SOFTWARE WORK PRODUCT VERSIONS 26
14.5. DOCUMENT VERSIONS 26

15. SOFTWARE RELEASE 26

15.1. SOFTWARE VERSION DESCRIPTION 26
15.2. SOFTWARE ARCHIVES 26

16. SOFTWARE QUALITY ASSURANCE RELATED PROCESSES 27

16.1. SOFTWARE VERIFICATION AND VALIDATION 27
16.2. QA RELATED ACTIVITIES 28

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

1. Purpose

This procedure provides the overall view of the activities of the software development life-cycle and defines the processes for the software development life-cycle in PeriGen. These software activities and processes support the development of software within the framework of the company. If a subcontractor participates in the development process or the software development is outsourced, the subcontractor shall either work according to this software development life-cycle procedure or according to an equivalent process, or the code received from the subcontractor shall be subject to review and testing by PeriGen personnel according to this procedure.

2. Scope

This procedure applies to all software development projects in PeriGen that are defined as medical device at the target market and are not exempted from QMS. This procedure applies to new designs and for changes made to existing products. Design and development projects performed by a subcontractor shall comply with PeriGen Design Control procedures and policies or with equivalent procedures.

3. Definitions and Abbreviations

The following are the acronyms used in this document:

1) API - Application Programming Interface
2) ALM - Application Lifecycle Management
3) CCB - Change Control Board
4) DHF - Design History File
5) DLL - Dynamic Linked Library
6) FDR - Feature Design Review
7) FDS - Feature Development Specification
8) HLD - High Level Design Description
9) IDD - Interface Design Document
10) MC - Micro Controller
11) OTS - Off The Shelf (software)
12) PO - Product Owner
13) QA/ RA - Quality Assurance and Regulatory Affairs
14) SCM - Software Configuration Management
15) SCR - Software Change Request
16) SDR - Software Design Review
17) SOP - Standard Operating Procedure
18) SOUP - Software Of Unknown Provenance
19) SPM - Software Project Manager

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

20) SPMP - Software Project Management Plan
21) SPR - Software Problem Report
22) SRR - Software Requirements Review
23) SRS - Software Requirements Specification
24) SSC - Software Safety Classification
25) SSO - Supplemental Services Order
26) STD - Software Test Description
27) STL - Software Team Leader
28) STP - Software Test Plan
29) STR - Software Test Report
30) SVD - Software Version Description
31) V&V - Verification and Validation
32) VVTL - Verification & Validation Team Leader

The following are the definitions of various terms used in this document:
Black box testing: Functional testing with no knowledge of the test object's internal structure.
Static analysis: Analysis of the software that is performed without executing the programs built from that software.
Walkthrough: A type of software peer review in which a software developer leads the members through a software product, allowing the members to ask questions and make comments about possible errors, violation of development standards, and other problems.
White box testing: Testing based on the unit's internal structure. The purpose is to exercise all paths through the code.

4. Applicable documents

The following are the documents referenced here.

Standards and Regulations**:**

[A1] General Principles of Software Validation; Final Guidance for Industry and FDA Staff, FDA, CDRH, 11/1/02
[A2] FDA Guidance for Off-the-Shelf Software Use in Medical Devices, 9/9/99
[A3] FDA Guidance for the Premarket Submissions for Software Contained in Medical Devices, 11/5/05
[A4] FDA 21CFR 820.30 Quality System Regulations
[A5] IEC 62304:2015 ed.1.1, Medical Device Software – Software Life-Cycle Processes
[A6] ISO 13485:2016, Medical Device Quality Management System, 7.3 Design and Development
[A7] EN ISO 14971:2012, Medical devices – Application of risk management to medical devices
[A8] AAMI TIR45:2012, Guidance on the use of AGILE practices in the development of medical device software

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

Procedures and Forms:

[B1] QMS-7.3.1, Design Plan procedure
[B2] QMS-7.3.2, Design Input procedure
[B3] QMS-7.3.4, Design Review procedure
[B4] QMS-7.3.7, Design Change procedure
[B5] QMS-7.3.8, Design Transfer and Release procedure
[B6] RD-WI-001, Risk Management
[B7] QMS-4.2.3, Documentation Controls
[B8] QMS-8.2.2, Internal Quality System Audits Procedure

5. Software Development Life Cycle

Management, marketing, design or external entity, may request the development or a substantial modification of a product as defined in Design Input Procedure (reference [B2]). The software is developed in accordance with the requirements defined. The software is developed, verified and validated according to this software development procedure. The software development procedures are based upon the software life cycle described in the IEC 62304 (reference [A5]) and supported by the FDA guidelines (references [A1] and [A3]).

The software development life cycle takes into account that many of the phases will be performed in iteratively or in parallel (following an Agile methodology where at any given time different stories can be in different stages of development) and one of the main purposes of this software life cycle procedure is to synchronize the software development phases with the developed products in such a way that the software is developed correctly, effectively and efficiently and that the PeriGen product is safe and effective.

5.1. Software Development Products

The software development can be in the form of a:

1) New product
2) Major modifications (re-engineering) of an existing product
3) New features added to an existing product
4) Bug fixes and minor changes
5) Utilities

Maintenance of the software is handled as a continuation of the software development life cycle where the maintenance can be attributed to major modifications (re-engineering) of an existing product, new features added to an existing product or bug fixes, or any combination.

The software can be developed for a desktop computer, smartphone or a tablet. The size and complexity of the software developed determines the amount of development activities and related documentation produced to verify and validate the software developed.

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

5.1.1. New Products

New products are designed and released in a controlled manner, and the product will require all software development activities as described below.

5.1.2. Major Modifications

Major modifications, defined here as changes that are not restricted to adding individual features to an existing product but rather changes that involve significant reengineering or rearchitecting of the product, will follow the same software development activities as defined for a new product.

5.1.3. New Features

Features are added to an existing product in a controlled manner. The upgraded product may contain one new feature, or a number of features developed simultaneously. Therefore, there are activities performed for each individual feature and there are activities performed on the product including the features developed.

5.1.4. Bug Fixes and Minor Changes

Bug fixes are usually performed when software bugs are discovered in a newly released software product and their fixes cannot wait for the next planned release. The bug fixes are performed on the released software product in a controlled manner. Minor changes usually are on the implementation level and are smaller in scope than features.

The major software activities for the bug fixes and minor changes are dependent upon the severity and extent of the bugs discovered and changes being introduced.

5.1.5. Utilities

Utilities are computer programs that perform routine tasks and supports operations, as distinct from the applications program. They may support the software development, production activities or clinical trials (among other applications).

The major software activities for the developing and maintaining the utilities are dependent upon the criticality and intended use.

Maintenance of the utility software is handled as a continuation of its software development. Depending upon the type and criticality of the changes to the utility software, the activities mentioned above can be performed as required.

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

5.2. Software development activities

Required activities vary depending on the form of software development project being undertaken:

	New product	Major modifications	New features	Bug fixes & minor change	Utilities
1) Hazard and risk analysis	✓	✓	✓	As required	As required
2) Software requirements	✓	✓	Version requirements	As required	As required
3) Top level design	✓	✓	Feature design	As required	As required
4) Detailed design	✓	✓	Feature design	As required	As required
5) Coding	✓	✓	✓	✓	✓
6) Unit testing	✓	✓	✓	✓	As required
7) Integration testing	✓	✓	✓	✓	As required
8) Verification testing	✓	✓	✓	✓	As required
9) Validation testing	✓	✓	✓	✓	As required
10) Clinical Validation	As required	As required	As required	As required	As required

In addition, the following software support activities are performed:

	New product	Major modification	New feature	Bug fixes & minor change	Utilities
1) Software project management	✓	✓	✓	✓	As required
2) Software configuration management	✓	✓	✓	✓	✓
3) Software reviews, walkthroughs and code reviews	✓	✓	✓	As required	As required
4) OTS validation	✓	✓	✓	N/A	As required
5) Software archiving and release	✓	✓	✓	✓	✓
6) Software process audits	✓	✓	✓	✓	✓

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

5.3. Software Development Phases

The software is developed using an Agile incremental/evolutionary lifecycle approach, based on the approach described in AAMI TIR45:2012 [A8].

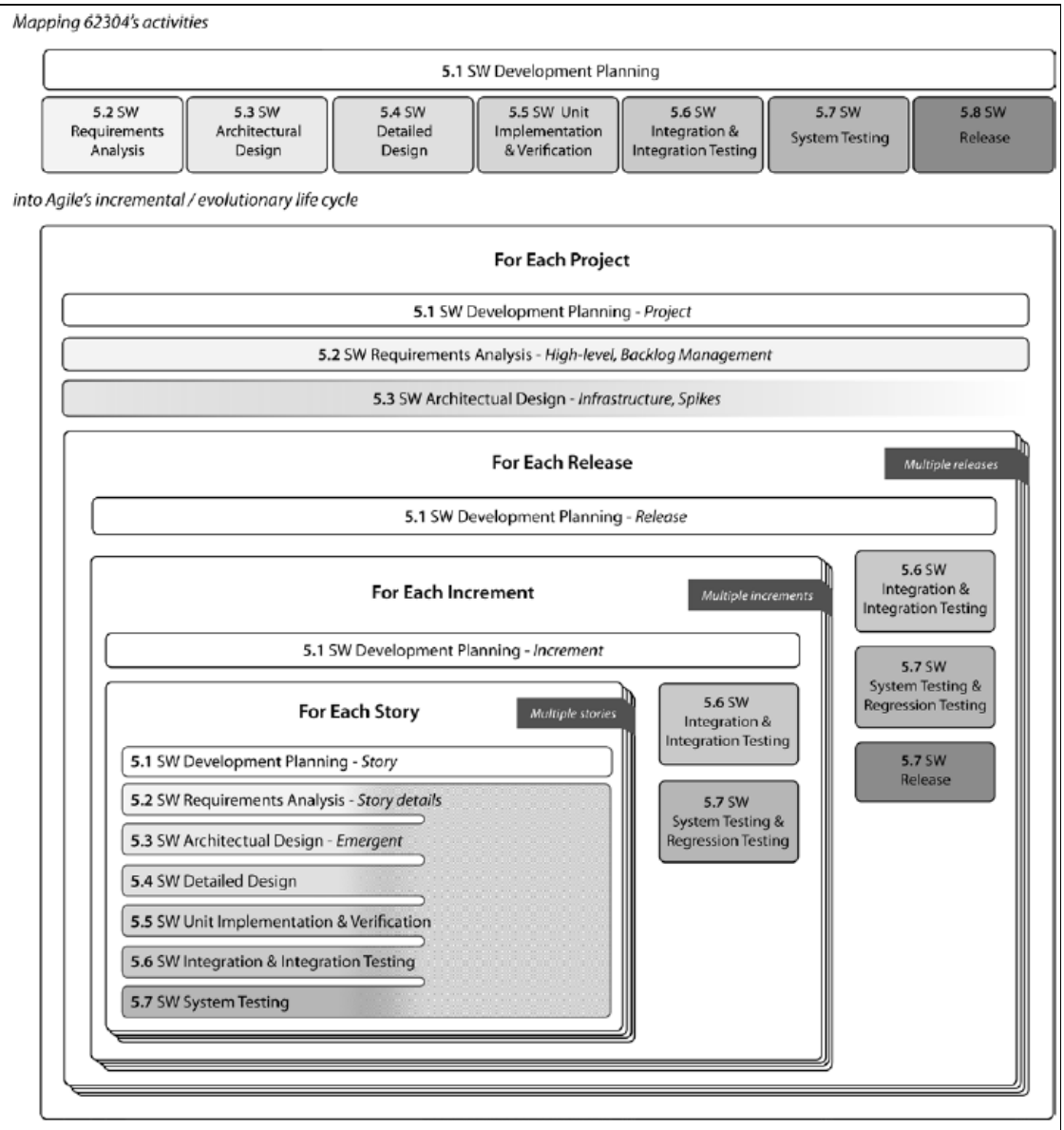

Figure 1 [Figure 4 from AAMI TIR45:2012]

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

Within each version release, development is divided into iterations, and the iterations divided into stories, with aspects of the software development lifecycle activities repeating as required for each iteration and/or story.

The level of formality and documentation required of these activites at each level is proportionate to the level – for example, at the story level, planning, requirements analysis, and design activities, is typically managed through informal "story breakdown" meetings with the breakdown then documented as tasks in the software ALM (Application Lifecycle Management) tool. Specific activities may be postponed to a "containing" level, e.g. system testing need not be carried out for each story but can be postponed for the iteration or release, as appropriate.

The software development outputs developed and maintained during the software life cycle for all types of software development are shown in the following table. The phases correspond to the software activities performed:

No.	Software Development Phase	Software Development Product	Section/SOP	Comments
1	Hazard and Risk Analysis	Risk Analysis	8 Software Risk Management	
2	Software Requirements	SRS	9 Software Requirements	
		VRS	Feature Development	Per version
3	Software Design	HLD	Software Design	Top level design
		FDS	Feature Development	Per feature
4	Detailed Design	Source Code, Code Review Summaries	Software/Feature Implementation	High level comments in the source code
5	Coding	Source Code, Code Review Summaries	Software/Feature Implementation	
6	Unit Testing	Source Code, Unit Test Summaries	Software/Feature Implementation	
7	Integration	Source Code, Integration Test Summaries	System Testing	
8	Verification Testing	Verification Test Summaries	System Testing	
9	Validation Testing Design	STP, STD	System Testing	
10	Validation Testing	STR, Test Logs	System Testing	Software Validation

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

11	Clinical Validation	Clinical Validation or clinical study, as appropriate	System Testing	Clinical Validation

Each phase is described in its corresponding section or procedure as shown in the table above.

5.4. Software Support Activities

The software support activities performed throughout the software life cycle for all types of software development are shown in the following table:

No.	Software Support Activity	Software Development Phases	Section/SOP	Comments
1	Software project management	All phases	6 Software Project Management	
2	Software configuration management	All phases	Software Configuration Management	
3	OTS validation	Validation Testing	OTS Software Validation	
4	Software archiving and release	After Validation Testing	Software Release	
5	Software process audits	All phases	Software Quality Assurance related Processes	Performed on the organization level

Each activity is described in its corresponding section or procedure as shown in the table above.

6. Software Project Management

The software project is to be managed such that PeriGen management can understand the needs (standards, tools, methods, resources, etc.), the schedule, the risks (non-safety) and are able to monitor the software project's progress. These issues are described in the Software Project Management Plan (SPMP).

The format for the SPMP is found in the SPMP template. The SPM is responsible for writing the SPMP. The SPMP will be updated as appropriate during the course of the project. The SPMP is a controlled document and all changes to the SPMP are to be handled as described in the Software Configuration Management Section.

New products will be initiated by creating a new SPMP for the project.

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

Software maintenance projects will be initiated by an Engineering Change Order (ECO) which will, depending on the complexity of the project, either require creation of a new SPMP for the project, or refer to and/or update an existing SPMP for the product and allow for its continued use for the maintenance project.

7. Level Of Concern/Software Safety Classification

The software Level of Concern (LOC) refers to an estimate of the severity of injury that a device could permit or inflict, either directly or indirectly, on a patient or operator as a result of device failures, design flaws, or simply by virtue of employing the device for its intended use. The LOC for the device software is determined according to the questions listed in the FDA Guidance for the Premarket Submissions for Software Contained in Medical Devices (reference [A3]). The LOC is determined at the beginning of the software development process.

The LOC is based on how the operation of the software associated with device function affects the patient or operator. The effect may be direct or indirect. The three levels are:

a) Major - if a failure or latent flaw could directly result in death or serious injury to the patient or operator, or if a failure or latent flaw could indirectly result in death or serious injury of the patient or operator through incorrect or delayed information or through the action of a care provider.

b) Moderate - if a failure or latent design flaw could directly result in minor injury to the patient or operator, or if a failure or latent flaw could indirectly result in minor injury to the patient or operator through incorrect or delayed information or through the action of a care provider.

c) Minor - if failures or latent design flaws are unlikely to cause any injury to the patient or operator.

The Software Safety Classification (SSC) is determined according to the requirements stated in IEC 62304 (reference [A5]). The SSC is assigned to each software system according to the possible effects on the patient, operator, or other people resulting from a hazard to which the software system can contribute. The Software Safety Classification assignment is performed during the start of the development and maintained throughout the software life cycle. The risk management and software architectural design are used in conjunction when determining the overall software safety classification of the medical device.

The safety classes are assigned according to the following:

The software system is software safety class A if:

– the software system cannot contribute to a hazardous situation; or

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

– the software system can contribute to a hazardous situation which does not result in unacceptable risk after consideration of risk control measures external to the software system.

The software system is software safety class B if:

– the software system can contribute to a hazardous situation which results in unacceptable risk after consideration of risk control measures external to the software system and the resulting possible harm is non-serious injury.

The software system is software safety class C if:

– the software system can contribute to a hazardous situation which results in unacceptable risk after consideration of risk control measures external to the software system and the resulting possible harm is death or serious injury.

When a software system is decomposed into software items, then each software item can be assigned its own software safety classification. The software architecture should promote segregation of software items associated with safety operations and risk control measures, and should describe the methods used to ensure effective segregation of those software items.

The Management Representative is responsible to conduct the LOC/SSC assessment related activities.

8. Software Risk Management

In every medical device there are potential hazards and risks to the patient and/or to the device operator. The risks or their mitigations may be determined by the software and directly affect its operation. The software risk management shall be performed as part of the device Risk Management as described in the Risk Management Procedure (reference [B6]), which is based upon the requirements described in ISO 14971 (reference [A7]). The specific aspects of the software risk management are performed in accordance with IEC 62304 (reference [A5]).

Requirements that are identified as risks in this process, and the features and stories derived from those requirements or which are mitigations to those risks, are flagged as such in the ALM (Application Lifecycle Management) tool and then receive additional focus during subsequent verification and validation testing.

9. Software Requirements

Management, marketing, design or external entity, may request the development or a substantial modification of a product as defined in Design Input Procedure (reference [B2]). The software of this product will be developed in accordance with the requirements defined. The software requirements are gathered based upon the design input requirements and recorded in the Software Requirements Specification (SRS). The risk mitigations implemented

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

in the software are to be traced from the Risk Analysis to the requirements in the SRS in the traceability matrix found in the SRS. The approved SRS forms the baseline for the software development and all software requirements' changes are controlled.

9.1. Software Requirements Specification

The SRS is written based upon the information from the following sources, among other sources, among other sources:

1) Device Specification
2) Design Input requirements
3) Risk, safety and hazard specifications
4) Technical specifications
5) Hardware specifications
6) 6) Regulatory Requirements

The format for the SRS is found in the SRS template

The PO is responsible for writing the SRS. The SPM is responsible to approve the SRS. The SRS is a controlled document and all changes to the SRS are to be handled as described in the Software Configuration Management Section. Changes to the SRS will be reflected in an update to the Risk Analysis in order to ensure that any changed risks are identified and evaluated.

9.2. Software Requirements Review

After the software requirements are identified for a new product, a Software Requirements Review (SRR) is held. The purpose of the SRR is to finalize the software requirements, to evaluate the software requirements for completeness, correctness, consistency, testability and traceability to the marketing and system requirements, and to initiate the software design phase. Relevant departments such as Management, Marketing , Operations or Development should be represented at the SRR. The STL is responsible for holding and summarizing the SRR, and tracking the action items to closure. The SRR summary is stored in the software section in the DHF.

During software maintenance, incremental changes to an existing SRS will be reviewed but may not require a formal SRR. Review and approval of the changes will be documented in the approval to the revised SRS.

9.3. Version Requirements Specification

For maintenance versions, a separate Version Requirements Specification (VRS) may be used to identify any new or changed requirements, rather than updating the original SRS. This is an optional method for maintaining the software where it is less convenient to update the SRS

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

for each version or feature. The determination whether to use a VRS or to update the SRS will be documented in the ECO.

The format for the VRS is found in the VRS template.

The VRS is a controlled document and all changes to the VRS are to be handled as described in the Software Configuration Management Section. Creation of a VRS and any changes to the VRS will be reflected in an update to the Risk Analysis in order to ensure that any changed risks are identified and evaluated.

The SRS will be periodically updated to reflect the cumulative changes in requirements found in sequential VRS, as appropriate. The timeframe for this is dependent upon the number, criticality and complexity of the requirements added to the system since the last SRS. Once the VRS is incorporated into the SRS, it becomes obsolete.

10.Software Design

The software design is comprised of top-level architectural design and detailed design. The top-level design determines the software architecture, behavior, interfaces and the logical structure of the database. The detailed design defines the internal structures and interfaces of and between the modules/classes/objects/packages/databases/algorithms/etc. The detailed design is handled as part of the software implementation.

The software architecture and the top-level design is recorded in the High Level Design Document (HLD). The software units designed are to be traced from the requirements in the SRS to the HLD using the traceability matrix found in the HLD. The approved HLD forms the baseline for the software implementation and all top-level design changes are controlled. The interfaces with external systems (if required) are documented in the Interface Design Document (IDD).

10.1. High Level Design Document

The HLD is written based upon the information from the following sources, among other sources:

1) Device Specifications
2) SRS
3) IDD/API
4) Risk, safety and hazard specifications
5) Technical specifications
6) Hardware specifications

The format for the HLD is found in the HLD template. The HLD reflects the top-level design. The detailed design is shown as high level comments in the source code. The database design can be incorporated in the HLD or in a separate Database Design Document. The STL is responsible for writing the HLD. The SPM is responsible to approve the HLD. The HLD is a

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

controlled document and all changes to the HLD are to be handled as described in the Software Configuration Management Section.

10.2. Interface Design Document

When the system interfaces with external systems or there are interfaces between different elements within the system, an IDD or API is required to describe the interfaces. If the IDD or API is the responsibility of the company it will be prepared and coordinated with the external system. If it is an IDD or API for an existing external system, the company shall develop the interface with the external system according to the IDD or API received from the external system.

If the IDD is to be prepared, the format for the IDD is found in the IDD template. The STL, in conjunction with the external systems, is responsible for writing the IDD. The IDD is a controlled document and all changes to the IDD are to be handled as described in the Software Configuration Management Section.

10.3. Software Design Review

After the software design is described, a Software Design Review (SDR) is held. The purpose of the SDR is to finalize the software design, to evaluate the software design for completeness, correctness, consistency, and traceability to the software requirements and to initiate the software implementation phase. Software managers, team leaders and other members of the development group should be represented at the SDR. The STL is responsible for holding and summarizing the SDR and tracking the action items to closure. The SDR summary is stored in the DHF.

10.4. Feature Design Specification

This is an optional method for maintaining the software where it is less convenient to update the HLD for each feature. Where features are being added incrementally to an existing product and do not significantly change the existing HLD, the features can be developed using the FDS approach rather than through HLD.

The feature development activities are comprised of the feature requirements identification and the feature design. The feature requirements are documented in the VRS for the version under development; the feature design determines its manner of implementation.

The feature design is recorded in the Feature Design Specification (FDS). The approved FDS forms the baseline for the feature implementation and all changes to the FDS are controlled. An FDS usually relates to only one feature but it may include a number of small or related features.

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

Each FDS that was implemented is incorporated into the baselined HLD every few versions. The timeframe for this is dependent upon the number, criticality and complexity of the features added to the system since the last HLD update. Once the FDS is incorporated into the HLD, it becomes obsolete.

The format for the FDS is found in the FDS template. The STL is responsible for writing the FDS. The SPM is responsible to approve the FDS. The FDS is a controlled document and all changes to the FDS are to be handled as described in the Software Configuration Management Section.

10.5. Feature Design Review

After the features requirements and design are described, a Feature Design Review (FDR) is held. The purpose of the FDR is to evaluate the FDS for correctness, completeness, consistency, testability and traceability to the system and marketing requirements, to finalize the feature design, and to initiate the feature implementation phase.

Feature Design Review may be carried out informally through discussions and emails and may not be documented in a separate FDR summary. Approval of the FDS includes a statement that the design has passed the necessary review.

When it is determined that a separate FDR summary document will be created, the SDR template shall be used for this purpose. The PO is responsible for holding and summarizing the FDR and tracking the action items to closure. The SPM is responsible to approve the FDR. The FDR summary is stored in the DHF.

11. Software/Feature Implementation

The software/feature implementation is comprised of the detailed design and coding of the software unit. The detailed design defines the internal structures and interfaces of and between the various software units. The coding translates the design into an executable program.

11.1. Detailed Design

The detailed design of the software unit is derived from the following:

1) SRS
2) FDS
3) HLD
4) IDD/API
5) Risk, safety and hazard specifications
6) Technical specifications
7) Hardware specifications
8) Programming environment (language, compiler, operating system, etc.)

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

The detailed design is developed either as design documents or as high-level comments in the source code. Each software developer is responsible for the detailed design of his software unit.

11.2. Walkthrough

Walkthroughs may be held to review and evaluate the detailed design where deemed necessary by the STL. The walkthroughs are held as peer reviews and can be performed in conjunction with the code reviews. The STL is responsible for conducting and summarizing the walkthroughs. When the walkthrough and the code review are performed together, only one summary is recorded. The walkthrough summary is stored in the DHF.

11.3. Coding

The software unit is coded according to the detailed design. The following may have an impact on the software implementation:

1) Hardware specifications
2) Programming environment (language, compiler, operating system, etc.)

The code is to be implemented according to the Coding Guidelines defined. Each software developer is responsible for the coding of his software unit. The STL is responsible for the code implementation by his team.

11.4. Code Review

Software units are to be reviewed according to their criticality, which is based upon their software safety classes as defined in IEC 62304 (reference [A5]) in combination with a consideration of the risk associated with a particular story as per section 8.

Where applicable, code review is performed as part of the approval process of “pull requests” to accept changes into the software code repository.

Additionally, each software developer developing code should have at least one unit reviewed periodically or major version to ensure that the coding guidelines are followed. The code is to be reviewed prior to integrating the software unit in the system. Only about 10% of the software units or features developed should be reviewed. The STL or another senior software developer reviews the code by reviewing it against the design and the coding guidelines. The STL is responsible for determining which software units to review and for conducting and summarizing the code reviews held. When the walkthrough and the code review are performed together, only one summary is recorded. The code review summary is stored in the DHF.

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

The code reviews can be performed using a static code analyzer. The coding rules should be defined for the static code analysis and a report should be prepared on the final run of the static code analysis. The static analysis report from these tests is stored in the DHF.

11.5. Unit Testing

Unit tests are performed on all software units developed. The purpose of the unit tests is to verify the software unit's code to its design. The unit tests may be performed informally by the software developer that developed the code. The unit tests are based upon white box tests running in the development environment.

Each software developer is responsible for the unit testing of his software unit. The STL is responsible for the unit tests on all the software units for his team.

For features developed for an existing system, the unit and integration tests can be performed concurrently in the same environment. In this case, only one set of test results is recorded. These combined unit and integration tests may be executed either by the software developer or by a software tester.

The unit test summary is stored in the DHF.

11.6. Code Control

Once the software unit is unit tested (and reviewed), the source code is put under software configuration management, as described in the Software Configuration Management Section. The STL is responsible for the configuration management of the software.

12. System Testing

The following system level tests are defined:

1) Integration tests
2) Verification tests
3) Validation tests

12.1. Integration Tests

Integration tests are performed during the integration of the software units within the system. The purpose of the integration tests is to verify the software units being integrated in the system with the existing software perform as expected. The integration tests are based upon a combination of white box and black box tests running in the development environment. The integration tests are performed informally by the development team. The functions to be tested are listed in a checklist. The functions found on the checklist that were tested and not tested are to be summarized.

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

The STL is responsible for the integration tests. For features developed for an existing system, the unit and integration tests can be performed concurrently in the same environment. In this case, only one set of test results is recorded. The integration test summary is stored in the DHF.

12.2. Verification Tests

The verification tests are performed after software passed the integration tests by the software development team. The purpose of the verification tests is to test the software to its limits and find as many defects as possible. The verification tests encompass the system tests, regression tests, load tests, monkey tests, stress tests and all other non-formal tests. The verification tests are performed informally by the V&V Team. The verification tests are black box tests running in the production environment. Additional white box tests can be run in the development environment. The functions to be tested in the verification tests are found in the checklists prepared for the tests.

The verification tests can be performed using the following tools:

1) Static Code Analysis
2) Dynamic Code Analysis

The VVTL is responsible for the verification tests and recording the verification test results. The verification test summary is stored in the DHF.

12.3. Software Validation Tests

The purpose of the validation tests is to validate the software's functionality and performance. The VVTL is responsible for the validation tests. The validation tests are performed after completing the software verification tests (after Design Freeze), and the software version is stable and is ready to be released (if the validation tests pass). The test strategy is described in the Software Test Plan (STP). The tests are described in the Software Test Description (STD).

12.3.1. Software Test Plan

The STP describes the scope of the validation tests required to validate the software version's functionality and performance. The scope of the validation tests is determined on a risk basis where the VVTL and PO decide on the scope required.

The format for the STP is found in the STP template. The VVTL is responsible for the STP. The SPM is responsible to approve the STP. The STP is a controlled document and all changes to the STP are to be handled as described in the Software Configuration Management Section.

The STP is not required if only one version of one product (and one validation) is handled at any given time. The STP shall be prepared where multiple versions or multiple products are handled simultaneously.

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

12.3.2. Software Test Description

The STD is written based upon the requirements and additional information found in the following sources:

1) SRS
2) VRS
3) FDS
4) HLD
5) IDD/API
6) Risk, safety and hazard specifications
7) User Manual
8) Technical specifications

The format for the STD is found in the STD template. The requirements implemented are to be traced from the requirements identified in the SRS to the test cases in the STD using the traceability matrix in the STD. The VVTL is responsible for the STD. The STD is a controlled document and all changes to the STD are to be handled as described in the Software Configuration Management Section.

12.3.3. Test Readiness Review

A Test Readiness Review (TRR) is held after the code has been completed and verification testing is nearing completion. Unless the TRR specifically decides otherwise, holding of a TRR constitutes a design freeze for the project. The purpose of TRR is to decide on the completion of the verification phase, approve the STP and STD, complete the setup for the software validation, and to initiate the software validation phase.

The TRR will review any remaining open issues, paying particular attention to any issues in areas that have been identified as being risk-related, and determine whether to proceed to validation.

Software managers, quality assurance, team leaders and other members of the development group should be represented at the TRR. The VVTL is responsible for the holding and summarizing the TRR and tracking the action items to closure. The TRR summary is documented on form SW-F-016 and stored in the DHF.

12.3.4. Validation Test Runs

The validation tests are run by the Testing Team and the test results are recorded on the test logs. Each test run is recorded with its run date, tester's name and pass/fail status for each step in the test procedure. If a test case is run more than once, each run will be recorded on a separate test log. The VVTL is responsible for the test runs and recording the results in the test logs. The test logs are stored in the DHF.

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

12.3.5. Clinical Validation

As part of design and development validation, PeriGen shall determine whether to perfor m clinical evaluation and performance evaluation as part of software validation, or whether a separate clinical study is required.

12.3.6. Software Test Report

The validation test summary is documented in the Software Test Report (STR). The format of the STR is found in the STR template. The VVTL is responsible for summarizing the test results in the STR. The STR is a controlled document and all changes to the STR are to be handled as described in the Software Configuration Management Section.

13. OTS Testing

There is a specific requirement that all Off The Shelf (OTS) software used in and by the software development be validated (reference [A2]). Accordingly, there are four types of OTS software used in the company:

1) OTS Source Code Component - This is a software component acquired together with its source code and is embedded in the software developed.
2) OTS Software Component - This is a run-time software component acquired that is used in the software product. The user cannot run it as stand-alone program. Examples of these may be the following file types: DLL, OCX, etc.
3) OTS Software Package - This is a software package that can be used by the user as a stand-alone program and is shipped along with the software product.
4) OTS Software for Use in the Office - This is a stand-alone software program that is used in the office for development and maintenance of the software products.

The OTS software is designed for general-purpose use and usually contains more functionality than is needed in the software products. The OTS software is functionally tested only for the functions that are used in the software products. Each new version of the OTS software requires a retest prior to integrating it within the software product. The VVTL is responsible to validate and approve all new OTS software and OTS software versions as defined in this procedure.

13.1. <u>OTS Source Code Component</u>

The OTS Source Code Component is a software component acquired together with its source code. Being that the OTS Source Code Component is compiled in the software product, it is treated in the same manner as any software unit that is developed in the company. The OTS

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

Source Code Component must satisfy all software product requirements. The OTS Source Code Component contains the same development lifecycle as developed software units.

13.2. OTS Software Component

The OTS Software Component is a run-time component (executable or library) acquired. It is treated in the same manner as any software unit that is developed in the company and contains the same development lifecycle as developed software units with the exception of the implementation phase. The OTS Software Component must satisfy all software product requirements. The functions to be incorporated in the software product are tested and are described in the System Test Description. The OTS Software Component is shipped as part of the software product.

13.3. OTS Software Package

The OTS Software Package is a stand-alone program and has to be tested accordingly. The tests for this OTS Software Package include stand-alone functional tests of those functions that are to be used with the software products. The OTS Software Package is also tested as part of the system test and validation, as described in STD document for the software product. If possible, a test case should be designed for each OTS Software Package in the STD. The system validation should be performed while the OTS Software Package is running.

In the case of complex software programs, e.g., operating systems or anti-virus software, the functionality is very difficult to test, and therefore, a justification of using the software should be given in place of the functional tests (citing, where possible, other organizations using this OTS software).

If applicable, the installation disk of the software and the software manufacturer's license and documentation should be shipped together with the software product.

13.4. OTS Software for Use in the Office

OTS Software for Use in the Office is a stand-alone software program that is used internally in the company for development and maintenance of the software products. This OTS Software for Use in the Office has to be tested according to its functionality and intended use.

In case of complex software, i.e., operating systems, software development environments, etc., where the functionality is difficult to test, a justification of using the software should be given in place of the functional tests (citing, where possible other organizations using this OTS software).

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

14.Software Configuration Management

This section documents the software configuration management (SCM) activities in the project. The STL is responsible for the software configuration management activities defined in this procedure, unless otherwise stated.

14.1. Configuration Identification

All software work products developed or purchased will be identified using the standard identification procedure found in the Documentation Controls procedure (reference [B7]).

The software work products (if applicable) include (but are not limited to):

1) Source code and executable files
2) Installation procedures and scripts
3) Development documents (SRS, HLD, FDS, IDD, etc.)
4) Test documents, data and results (STD, STR, etc.)
5) Release documents (SVD, release notes, user manuals, etc.)

14.2. Configuration Control

Configuration control describes the process used in controlling changes to and maintaining the software work product configurations. The change control is performed in accordance with the Design Change procedure [B4]). The software work products include all controlled documents and software code.

Planned releases based upon the functionality found in the Roadmap and the SSOs are planned and described in the SPMP or ECO.

Software changes and problems for resolution are evaluated by the PO in consultation with the CCB to determine the need for the change and the resulting impact. The PO maintains a list of Software Change Requests (SCR) and Software Problem Reports (SPR) in electronic form in an overall product backlog, and approves or rejects the change requests based on their relative priority in combination with a consideration of the likely development cost of each item and its relationship to the overall Product Roadmap.

Once approved for inclusion in a product release, the relevant SCRs and SPRs are detailed in the ECO for the designated version and identified as stories or bugs in the ALM tool for further tracking until resolution as part of the normal software development process.

The list of SCRs and SPRs for inclusion in the release may be further modified as appropriate, based on changing priorities or scheduling constraints. Such changes will be handled as revisions to the ECO.

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

14.3. Configuration Management Libraries

All approved software work products are stored in the configuration management library for the project. Documents are stored in project specific, read-only directories and the DHF.

14.4. Software Work Product Versions

The standard version number is not an integral part of the configuration item identification and is separate from the identification. The version number convention is defined in the relevant SPMP.

14.5. Document Versions

Versioning of the controlled documents is handled as described in the Documentation Controls procedure (reference [B7]).

15.Software Release

The software product developed in the company that is transferred to an external entity shall be released in a controlled manner. The transfer of the software product can be as a completed product, or as a subsystem of a larger product, or as a software prototype used for demonstration at external sites. The software product is released only after it has completed the software verification and software validation processes. Release to market is performed only after the approval of the QA/ RA Manager according to the Design Transfer and Release procedure (reference [B5]).

15.1. Software Version Description

Only after the software product has completed the software verification and validation processes, the Software Version Description (SVD) is prepared. The format for the SVD is found in the SVD template. The STL is responsible for writing the SVD. The SPM is responsible to approve the SVD.

15.2. Software Archives

After completing the SVD, the software work products for this software version are stored in the software archives. The software work products (if applicable) to be stored in the software archives include (but not limited to):

1) Source code and executable files
2) Installation procedures and scripts
3) Development documents (SRS, HLD, FDS, IDD, etc.)
4) Test documents, data and results (STD, STR, etc.)

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

5) Release documents (SVD, release notes, User Manual, etc.)
6) Development environment
7) OTS software
8) SOUP

The STL is responsible for preparing and storing the software work products in the software archives.

16. Software Quality Assurance Related Processes

The verification and validation activities and the quality assurance for the software shall be performed within the framework of the software development. Verification and validation activities relate to the software work products. Software quality assurance activities in the project relate to the software development processes.

16.1. Software Verification and Validation

All the software work products developed during the software life cycle are to be verified and the final product validated. The following table depicts the methods employed to verify the software work products.

No.	Phase	Output	Verification Method	Responsible	Comments
1	Project Planning	SPMP	Review	SPM	
		ECO	Review	SPM	
2	Risk Analysis	RMF	Review	PO	See Risk Management Procedure
3	Software Requirements Analysis	SRS	Review (SRR)	PO	
4	Version requirements	VRS	Review	PO	
5	Feature Design	FDS	Review	PO	In conjunction with the STL
6	Software Design	HLD	Review	STL	
7	Software/Feature Implementation	Code	Code Review + Unit Tests	STL	Includes detailed design, coding and unit testing

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.

No.	Phase	Output	Verification Method	Responsible	Comments
8	Software/Feature Integration	Integration Tests Summaries	Integration Tests	STL	
9	Verification Testing	Verification Tests Summaries	Verification Tests	VVTL	
10	Software Validation Test Planning	STP	Review	VVTL	In conjunction with the PO
11	Software Validation Test Design	STD	Review (TRR)	VVTL	
12	Software Validation Testing	STR	Software Validation Tests	VVTL	Software Validation
13	Clinical Validation				Clinical Validation, as applicable
14	Software release	SVD	Review		

The SPM is responsible for monitoring the software verification and validation process as part of the testing activities.

16.2. QA related Activities

Software quality assurance relates to the set activities that support the quality aspects of the development processes. The QA Manager should monitor the software development process via the use of audits. When discrepancies are discovered in the software development process a warning is issued. The software process audits are held in conjunction with the company' internal audit plan as described in the Internal Audits Procedure (reference [B8]).

©PeriGen, Inc. – Confidential and Proprietary
This document may not be copied without express written permission.